

EncGradInversion: Image Encoding and Gradient-Inversion-Based Batch Attack in Federated Learning

Thi-Nga Dao^{1b} and Hyungjune Lee^{2b}, *Member, IEEE*

Abstract—The gradient attack problem has recently been studied to increase the awareness of people on privacy risks in federated learning. However, this attack is constrained under specific conditions, such as small image batch sizes and low image resolutions. To address this challenge, we introduce a new three-phase image recovery architecture called *EncGradInversion*, which harnesses the power of image encoding and the shared gradient inversion. In the first phase, we attempt to extract the representation for all of the images using the gradient at the final layer. Then, in the second phase, the extracted encoding of a specific image is leveraged for reconstructing the image by matching the representation of dummy and approximated images. This allows a parallel algorithm to accelerate the image recovery. In the final phase, the reconstructed images are fine tuned using the shared gradient of the whole network. In the second and third phases, we formulate an optimization problem to minimize the discrepancy between the shared and reconstructed gradients, while preserving the smoothness and natural appearance of the reconstructed images. Evaluated on various datasets and deep learning models, *EncGradInversion* shows its superiority to recover the original training images with resolutions as high as 1024×1024 and with the batch size of 512. Furthermore, the proposed architecture outperforms existing counterparts with a factor of up to 9.8 and 6.04, in terms of structural similarity performance and attack time.

Index Terms—Federated learning (FL), gradient inversion, gradient leakage attack, image recovery.

I. INTRODUCTION

DUE TO the emphasis on privacy and decentralization preservation, federated learning (FL) has widely attracted both researchers and practitioners across various industries [1], [2]. Data, such as private images and texts collected from Internet of Things (IoT) devices [3], [4], [5], [6] or financial data are used to train a global model for a wide range of classification and regression tasks. In FL, each participant computes the gradient of parameters using a

training batch and shares the gradient with other participants or a central parameter server. However, a critical concern in FL lies in the potential attacks from adversaries who attempt to recover sensitive raw training images from the shared gradients, called gradient attacks. Existing works show that the recovery quality highly depends on the batch size and image resolution. More specifically, the reconstructed images have relatively poorer quality as the batch size or image resolution increases. Therefore, investigating a more effective attack strategy to obtain high-quality images under large batch sizes and image resolutions remains interesting but challenging.

To reconstruct the training images from the shared gradient, a common method is to form an optimization problem that minimizes the discrepancy between the shared and dummy gradients [7], [8], [9], [10], [11], [12], [13], [14], [15], [16]. The dummy images treated as variables of the optimization problem are trained to minimize the objective function. To further enhance the attack quality, different regulation terms are added to the objective function. For example, total variation loss is used to increase the smoothness of reconstructed images, while the norm is used to constrain the reconstructed images within a feasible range from 0 to 1. Besides the shared gradients, it was validated that the underlying training inputs can be recovered using the image representation from an intermediate hidden layer. The reconstruction quality highly depends on the distance between the input layer and the intermediate layer, i.e., the higher performance can be obtained with a closer distance.

Motivated by existing works that the training images can be revealed using the shared gradient or the image representation, we propose a novel attack framework called *EncGradInversion* that recovers the image batch by leveraging the interplay of image encoding and gradient inversion, to achieve high-quality images under large batch sizes and high resolutions. It should be noted that since the image representation is not shared by participants in FL, we first design a method to approximate the image encoding at the final hidden layer for each training image. Then, each training image is reconstructed separately using the approximated image representation. Finally, all of the images are trained to minimize the difference between the shared and recovered gradients, while keeping the image natural and keeping images within a limited range.

We design two essential components for image representation reconstruction called one-hot activation (OHA) and gradient average (GRA) in the first phase. The OHA method assumes that only one training image with label j contributes to the gradient from the final hidden layer related to output j .

Received 11 May 2024; accepted 7 October 2024. Date of publication 21 October 2024; date of current version 6 February 2025. This work was supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) Grant funded by the Korea Government (MSIT) under Grant RS-2022-00155966; and in part by the Artificial Intelligence Convergence Innovation Human Resources Development (Ewha Womans University) Artificial Intelligence Innovation Hub under Grant RS-2021-II212068. (*Corresponding author: Hyungjune Lee.*)

Thi-Nga Dao is with the Faculty of Radio-Electronic Engineering, Le Quy Don Technical University, Hanoi 10000, Vietnam (e-mail: daothinga.mta@gmail.com).

Hyungjune Lee is with the Department of Computer Science and Engineering, Ewha Womans University, Seoul 03760, Republic of Korea (e-mail: hyungjune.lee@ewha.ac.kr).

Digital Object Identifier 10.1109/IIOT.2024.3483850

Based on the characteristics of the backpropagation algorithm, we derive the image encoding from the gradient in the final hidden layer. Meanwhile, in GRA, we approximate the gradients corresponding to the nontrue outputs as the average value of softmax output units. Then, a sequence of linear equations with variables of image encodings are formulated, and the image representations can be obtained by solving these linear equations.

In the next phase, we formulate a weighted loss function that represents the difference between the approximated and dummy image representations, the total variation loss, and the norm regularizer. Each training input is initialized randomly and then updated using the L-BFGS optimizer to match the approximated and dummy gradient while maintaining the smoothness and naturalness of recovered images. Since a training image can be recovered independently in this phase, a parallel algorithm can be used to accelerate the recovery process. Moreover, provided that image encodings are recovered accurately, the performance of image reconstruction does not depend on batch size, which is mainly different from existing works.

In the final phase, we further enhance the recovered images by matching the shared and reconstructed gradients across all of the layers. A weighted loss function is formulated by computing the difference between real and reconstructed gradients from all of the layers, the smoothness, and the naturalness of the reconstructed images. Unlike the second phase where each training image is updated in parallel, in the third phase, all of the training images are jointly learned to minimize the weighted loss function. Experimental results show that the second phase contributes to obtaining a nearly complete recovery of the training images, while the third phase serves to further enhance and fine tune the reconstructed image batch.

This work is the first attempt to recover training images with high batch sizes and high-resolution images by effectively leveraging both image encoding and shared gradients. We conducted extensive experiments for the image batch attack using various well-known datasets of CIFAR-100, LFWPeople, and ImageNet and various deep convolutional neural networks (CNNs), such as LeNet, ResNet-18 [17], and MobileNet [18]. Evaluation results show that *EncGradInversion* significantly outperforms existing works in terms of reconstruction quality and attack time especially when the batch size increases. *EncGradInversion* is capable of reconstructing images with resolutions of as high as 1024×1024 , with a batch size of 512, which can pose a real critical threat in FL.

The main contribution of our work can be summarized as follows.

- 1) We introduce a novel three-phase image batch attack framework that makes use of image encoding and shared gradient for the reconstruction of training images with high resolution and high batch size in distributed learning.
- 2) We propose an effective way of image representation reconstruction that approximates the representation of each training image from the gradient in the final hidden layer. Then, image encoding is leveraged to

recover each training image independently, allowing the reconstruction of images to be performed in parallel.

- 3) We formulate a weight loss function that measures the difference between the gradients of the shared and reconstructed images as well as the realism of reconstructed images. Then, the image batch is found as an optimal solution by minimizing the loss function.
- 4) The results obtained from experiments on various datasets and network architectures demonstrate the significant effectiveness of incorporating both image encoding and gradient for the image batch attack. Our finding shows that image encoding extracted from the final layer is more useful information for image batch attacks than the shared gradient from all of the layers.

This article is organized as follows. Section II summarizes the existing works in the literature and compares them with our method. Then, the main assumptions and the threat model are presented in Section III, and then we describe our main image recovery algorithms in Section IV. The experimental results of *EncGradInversion* are reported and analyzed in Section V. Finally, we conclude this work with some possible future works in Section VI.

II. RELATED WORK

Since our work is related to gradient attacks, we compare existing research studies as summarized in Table I. We list the main features of related studies, including shared information, required knowledge, method type, attack type, inversion information used for image reconstruction, optimizer, image resolution, batch size, and published year. We denote $\partial \mathbb{L} / \partial \theta$ as the gradient with respect to network parameters θ . In auxiliary free, the attacker is assumed not to include any auxiliary samples, whereas, in auxiliary-based methods, the attacker is required to have an auxiliary dataset that is used to achieve better dummy image initialization instead of random initialization. In addition, the existing works can be classified into optimization-based or analytics-based types and passive or active attacks. Specifically, reconstructed images are the solution of an optimization problem in the optimization-based method or have closed-form expressions in the analytics-based method. Unlike the passive attack, the attacker has the privilege to change the network architecture and network parameters in the active attack.

Deep leakage from gradient (DLG) in [7] is the first work that formulates an optimization problem to reconstruct the dummy images and dummy labels from the shared gradient. As variables in the optimization problem, dummy images and labels are randomly initialized and then updated in a way that the dummy gradient is as close as possible to the shared gradient. The mean-squared error measures the difference between dummy and shared gradients. Even though the results look promising, this method can only recover images with a limited batch size of 8 and an image resolution of 64×64 .

To improve the performance of DLG, Geiping et al. [8] added a regularization term into the loss function and used cosine similarity to measure how the dummy gradient is similar to the shared gradient. The regularizer includes a

TABLE I
SUMMARY OF EXISTING AND OUR WORK

Paper	Shared Info	Knowledge	Method Type, Attack Type	Inversion	Optimizer	Image Resolution	Batch Size	Year
[7]	$\theta, \partial\mathbb{L}/\partial\theta$	Auxiliary-free	Optimization, passive	Gradient	L-BFGS	64x64	8	2019
[8]	$\theta, \partial\mathbb{L}/\partial\theta$	Auxiliary-free	Optimization, passive	Gradient	Adam	32x32	100	2020
[9]	$\theta, \partial\mathbb{L}/\partial\theta$	Auxiliary-free	Analytics, passive	-	-	32x32	2	2021
[10]	$\theta, \partial\mathbb{L}/\partial\theta$, BN	Auxiliary-free	Optimization, passive	Gradient	Adam	224x224	48	2021
[11]	$\theta, \partial\mathbb{L}/\partial\theta$	Auxiliary-free	Analytics, active	Gradient	-	224x224	512	2022
[12]	$\theta, \partial\mathbb{L}/\partial\theta$, BN	Auxiliary-free	Optimization, passive	Gradient	-	256x256	512	2023
[13]	$\theta, \partial\mathbb{L}/\partial\theta$	Auxiliary-free	Analytics, active	Gradient	-	224x224	100	2023
[14]	$\partial\mathbb{L}/\partial\theta$	Auxiliary-free	Optimization, passive	Gradient	Gradient descent	32x32	32	2023
[15]	θ , compressed $\partial\mathbb{L}/\partial\theta$	Auxiliary-based	Optimization, passive	Gradient	Adam	32x32	16	2023
[16]	$\partial\mathbb{L}/\partial\theta$ in FedAVG	Auxiliary-based	Optimization, passive	Gradient	L-BFGS	256x256	128	2023
Our work	$\theta, \partial\mathbb{L}/\partial\theta$	Auxiliary-free	Optimization, passive	Gradient, image encoding	L-BFGS	1024x1024	512	-

BN: Batch normalization statistics

simple image prior called total variation loss to denoise the reconstructed images. In addition, the Adam optimization algorithm is considered to update dummy images.

Recursive gradient attack on privacy (R-GAP) [9] formulated the gradient attack as a set of linear equations and explained when DLG produces a unique image recovery for the optimization problem. Then, they proposed a recursive method to find the closed-form expression of reconstructed images by solving linear equations. Although R-GAP can produce higher image reconstruction, this method does not work well when the batch size is greater than 1.

In see through gradient (STG) [10], the authors formulated an optimization task to match gradients, while ensuring image fidelity by adding a total variation and norm loss into the loss function. Moreover, instead of optimizing one possible candidate for a training image, each agent generates a group of candidates for a training image. Then, the group consistency loss is added into the loss function as a regularization term to push all candidates to converge to a consensus image. By appropriately selecting coefficients of the loss function, STG can recover the training image with a resolution up to 224×224 . However, STG requires batch-wise mean and variance at batch normalization (BN) layers, which are not usually shared between agents.

Unlike [7] and [8], Hatamizadeh et.al investigated how statistics of BN layers affect the gradient attack in [12]. The loss function now contains gradient loss, image prior loss, and regularization loss. While the first two losses are similar to that in [10], the BN regularizer measures the difference between BN statistics (mean and variance) of dummy and real inputs. Although using BN statistics can help to enhance the reconstruction quality, sending personalized BN statistics specific to other clients is not a practical assumption. In this work, we assume that only model parameters and model architecture are shared between clients in FL and the adversary does not know BN statistics.

In [11], the work assumes that the attacker has the capability of modifying the model architecture and model parameters. From the analytical theory that the input of any hidden layers can be recovered fully at the beginning of training, it added a fully connected network to the original model at the position that is close to the input layer. After modifying the network architecture, the training images can be recovered

much more easily than passive attacks. However, the normal agent may also recognize the change in network architecture. Reference [13] launched an active gradient attack that requires not only the change in weight distribution but also network parameters. The weight initialization aims to keep only one output neuron active for one input image per mini-batch. Therefore, the success rate greatly depends on the adversarial weight initialization. The experimental results show a success rate of around 0.4 to 0.5.

Unlike other works that assume either a server or a client is compromised, a recent work called external gradient inversion attack (EGIA) [14] considers that both the server and the clients are fully trusted and proves that the gradient attack at an intermediate node is possible. In EGIA, the attacker does not need to know the network parameters and relies only on the gradient shared by the server. More specifically, the intermediate node can send dummy images to the server and receive the dummy gradient. Then, they can derive the difference between dummy and real gradients. The recovery results show that images with a resolution of 32×32 and a batch size of 32 can be almost recovered.

Geng et al. [16] proposed an attack method with *FedAvg* in which parameters are trained multiple rounds, and then, the aggregated gradient is shared with the adversary. A simple strategy was adopted by approximating the gradient at each round as the average value of the aggregated gradient. The suggested method can recover batch images with a batch size equal to 64. However, the quality of recovered images highly depends on the number of training rounds which images are trained for. In terms of the objective function, a gradient loss and three different auxiliary regularizers are used: 1) total variation; 2) clip; and 3) scale. While the total variation loss can help to denoise images, clip and scale losses make sure that values of reconstructed images are in the range of [0, 1].

Yang et al. [15] proved that the gradient attack can still successfully recover the training images by using the compressed gradient. They proposed a network generator that initializes dummy images using the gradient from the final layer. The loss function is formulated to represent the discrepancy between the compressed dummy gradients and the compressed real gradients. Finally, a denoising model is adopted to further improve the reconstruction quality.

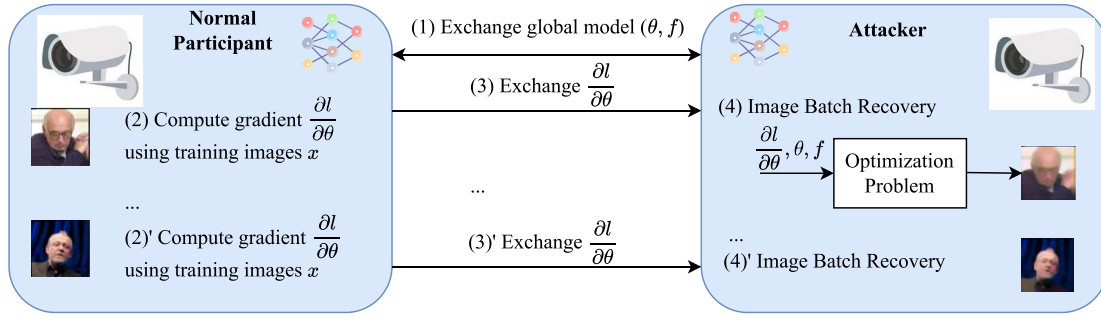


Fig. 1. Gradient attack in peer-to-peer FL.

Our work addresses a more challenging and practical gradient attack problem with large batch sizes and high image resolutions. The main limitation of existing works of passive attacks is that they can only recover training images with low image resolution and small batch size. Moreover, there does not seem to exist any work on accelerating the gradient attack by separately recovering each training image in the batch. Our *EncGradInversion* aims to address these limitations by leveraging both image encoding and the shared gradient for reconstructing the training images.

III. THREAT MODEL

We consider a gradient leakage model in FL in Fig. 1. The adversary can be a parameter server in a centralized FL or any participant in a peer-to-peer FL, such as a specific customer, user, or hospital who has their private training data. Each participant in the FL model is equipped with an IoT device, such as mobile phone, computer, industrial sensor, or camera. Each IoT device leverages local computation resources to train its local model using its dataset. Clients in FL aim to jointly build a global classification model for one of the following tasks: object classification with the datasets like CIFAR-10 or ImageNet, face recognition with the dataset like LFWPeople, and disease diagnosis with the datasets like chest X-ray or brain MRI images. For example, in Fig. 1, two participants collect training images from smart cameras and then train a face recognition model using FL. We assume that the global homogeneous model contains a list of CNN layers followed by a hidden layer and the softmax output layer. In each training epoch, the victim updates network parameters using an image batch with a batch size greater than or equal to 1. Based on the shared gradient from the normal device, the attacker launches the gradient attack to reconstruct the training images. The gradient leakage model consists of four main steps as follows.

- 1) Both participants share the initial global model represented by network architecture f and network parameters θ .
- 2) The IoT device from a normal party computes the output $y = f(x, \theta)$ given training images x and then derive the gradient $(\partial l / \partial \theta)$.
- 3) The victim device sends $(\partial l / \partial \theta)$ to the adversary.
- 4) The attacker aims to recover the private training images by finding the solution for the following optimization

problem:

$$\text{minimize } \mathbb{L} = g(x, \theta, f) \quad (1)$$

where $\mathbb{L}$ is the loss value that depends on the recovered images x , network parameters θ , and network architecture f . In this work, we define the loss value as

$$\mathbb{L} = L_g + L_h + L_{tv} + L_{\text{norm}} \quad (2)$$

where $L_g = ((\partial l / \partial \theta) - (\partial l^* / \partial \theta))^2$ is the difference between the shared gradient and dummy gradient; $L_h = (h - h')^2$ is the difference between the dummy encoding and real encoding at the final hidden layer; L_{tv} is the total variation loss that is added as a regularizer to maintain the fidelity of images; and L_{norm} is another regularizer used to constrain the images value within a specific range.

The gradient attack will be continued in another round starting from steps 2 to 4 for a new training batch. We now explain why L_h is added to the loss function. Several studies [19] found that it is possible to guess the training inputs using the image encoding at an intermediate hidden layer. Better reconstruction performance can be produced when the distance between the encoding and the input layers is closer. Motivated by the finding in [19], the loss term L_h is added in the loss function together with L_g and two regularization terms L_{tv} and L_{norm} , as in 2.

IV. THREE-PHASE IMAGE BATCH RECOVERY ALGORITHM

In this section, we present an image batch recovery algorithm that consists of three phases: 1) image encoding reconstruction; 2) image reconstruction using image encoding; and 3) image reconstruction using the whole shared gradient. The overview of the proposed architecture is shown in Fig. 2. First, image representation at the final hidden layer is reconstructed using the shared gradient at the final hidden layer. Then, in the second phase, the reconstructed images are obtained by solving the optimization problem that minimizes the difference between the reconstructed and dummy representation. Finally, the reconstructed image is fine tuned by matching the shared and dummy gradients and matching the reconstructed and dummy encoding. In the second and third phases, two kinds of regularizers are leveraged, including total variation and norm loss. Table II summarizes the notations used in this article.

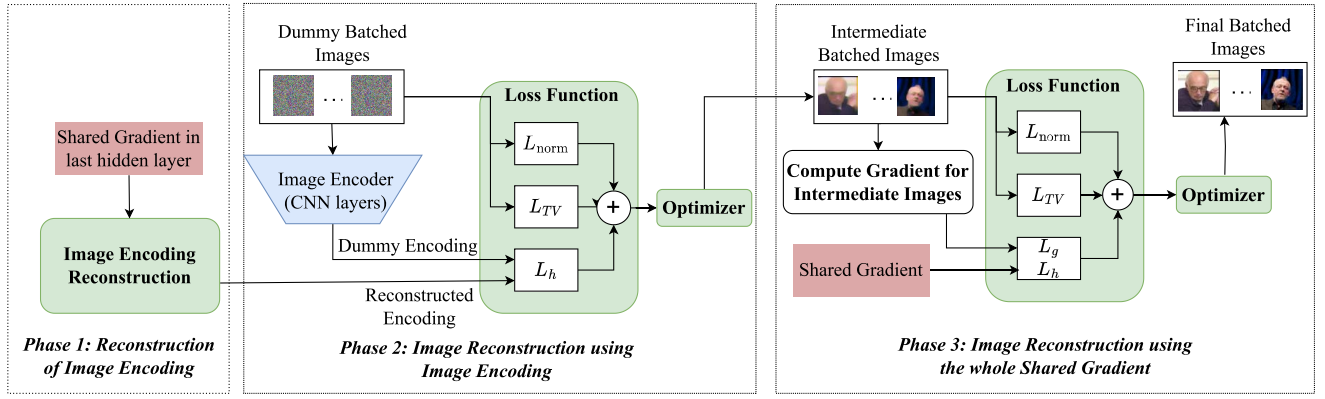


Fig. 2. Overview of three-phase image batch recovery framework *EncGradInversion*.

TABLE II
NOTATIONS

Name	Description	Note
B	Batch size	$B \geq 1$
C	#Classes	
n_h	#Hidden units	
$\mathbf{x}$	Original images	Size: B
$\mathbf{x}^*$	Reconstructed images	Size: B
i	Image index	$i = \{1, 2, \dots, B\}$
j	Output index	$j = \{1, 2, \dots, C\}$
$\partial l / \partial W_{jk}$	Gradient wrt weight W_{jk}	
$\partial l / \partial b_j$	Gradient wrt bias b_j	

A. Image Encoding Reconstruction

The softmax output layer can be represented as

$$y = \text{Softmax}(hW + b) \quad (3)$$

where vector h represents the final hidden layer. Let $z = hW + b$ denote the output values before applying the softmax activation function. $h_k^{(i)}$ is the hidden unit k for image i . According to the chain rule of derivative for a compound function, $(\partial l / \partial W) = (\partial l / \partial z)(\partial z / \partial W) = (\partial l / \partial z)h$. That means the gradient in the final layer for image i is the production of $h_k^{(i)}$ and $(\partial l / \partial z_j^{(i)})$. With the batch size $B > 1$, the shared gradient in the final layer is averaged over the image batch as follows:

$$\frac{\partial l}{\partial W_{kj}} = \frac{1}{B} \sum_{i=1}^B h_k^{(i)} \frac{\partial l}{\partial z_j^{(i)}} \quad (4)$$

$$\frac{\partial l}{\partial b_j} = \frac{1}{B} \sum_{i=1}^B \frac{\partial l}{\partial z_j^{(i)}} \quad (5)$$

where W_{kj} represents the connection weights between hidden neuron k and output neuron j and b_j is bias of output j . The cross-entropy loss l for image i with true class c is computed as: $l = -\log(e^{z_c^{(i)}} / \sum_j e^{z_j^{(i)}})$. The gradients with respect to z for image i are derived as follows:

$$\frac{\partial l}{\partial z_j^{(i)}} = \begin{cases} -1 + \frac{e^{z_j^{(i)}}}{\sum_j e^{z_j^{(i)}}}, & \text{if } j = c(\text{class index of } x_i) \\ \frac{e^{z_j^{(i)}}}{\sum_j e^{z_j^{(i)}}}, & \text{if } j \neq c \end{cases} \quad (6)$$

where c is the class label of image i . Then, we propose two methods to approximate the image representation. The first method is called OHA or OHA since we assume that only image i will activate the gradient connecting to the output unit with the label of that image. The second method is called GRA since we assume that $(e^{z_j} / \sum_j e^{z_j})$ has a close value of μ and μ is the average value of $(e^{z_j} / \sum_j e^{z_j})$ with $j \neq c$.

Method 1 (OHA): Assume that there are a large number of classes, i.e., $(e^{z_c} / \sum_j e^{z_j})$ is quite small. Therefore, $(e^{z_c} / \sum_j e^{z_j})$ can be ignored at the beginning of the training period and only one image in the training batch contributes to the shared gradient in (4) and (5). Then, the image encoding for a specific image can be derived as

$$h_k^{(i)} = \frac{\partial l}{\partial W_{kc}} / \frac{\partial l}{\partial b_c} \quad (7)$$

where index $k = 1, 2, \dots, n_h$, and n_h is the number of hidden units in the final hidden layer. Note that, existing work [20] proved that the true labels of training data can be successfully predicted. Therefore, it is possible to recover image encoding from the shared gradient in the final hidden layer, especially at the beginning of training.

Method 2 (GRA): In this method, we assume that $(e^{z_j^{(i)}} / [\sum_j e^{z_j^{(i)}}])$ with $j \neq c$ has a similar value of μ , (5) can be expressed as:

$$(\partial l / \partial b_j) = (1/B)[-1 + (e^{z_c} / [\sum_j e^{z_j}]) + \mu(B-1)].$$

We derive $(e^{z_c} / [\sum_j e^{z_j}])$ as a function of $(\partial l / \partial b_j)$, μ , B , and then (6) can be rewritten as

$$\frac{\partial l}{\partial z_j^{(i)}} = \begin{cases} B \frac{\partial l}{\partial b_j} - \mu(B-1), & \text{if } j = c(\text{class index of } x_i) \\ \mu, & \text{if } j \neq c. \end{cases} \quad (8)$$

By substituting coefficients $(\partial l / \partial z_j^{(i)})$ into (5), we have a set of B linear equations with B variables $h_k^{(i)}$ where $i = 1, 2, \dots, B$. The approximated value of $h_k^{(i)}$ can be found by solving these linear equations. Algorithm 1 summarizes the GRA method for the image representation reconstruction. Specifically, hidden representation can be reconstructed iteratively using a for loop from lines 2 to 7. Now, we explain the representation reconstruction of specific hidden unit k . At line 2, we construct vector b with B elements $[(\partial l / \partial W_{k1}), (\partial l / \partial W_{k2}), \dots, (\partial l / \partial W_{kB})]^T$. Then, the matrix A

Algorithm 1 GRA Method

Input: Network architecture, network parameters (weight, bias), shared gradient wrt parameters on batched data in the last hidden layer

- 1: **for** $k = 1, 2, \dots, n_h$ **do**
- 2: $b = \left[\frac{\partial l}{\partial W_{k1}}, \frac{\partial l}{\partial W_{k2}}, \dots, \frac{\partial l}{\partial W_{kB}} \right]^\top$ $\triangleright$ Construct vector $b \in \mathbb{R}^{B \times 1}$
- 3: **for** $j = 1, 2, \dots, B$ **do**
- 4: $A[j, :] = \left[\frac{\partial l}{\partial z_j^{(1)}}, \frac{\partial l}{\partial z_j^{(2)}}, \dots, \frac{\partial l}{\partial z_j^{(B)}} \right]$ $\triangleright$ Construct matrix $A \in \mathbb{R}^{B \times B}$
- 5: **end for**
- 6: Formulate and solve linear equations: $A \times h_k = b$
- 7: $H[k, :] = h_k$ $\triangleright$ Solution h_k is representation of images at unit k
- 8: **end for**

Output: Image representation $H \in \mathbb{R}^{n_h \times B}$

with shape of $B \times B$ is formed as shown in lines 3 to 5. The j th row of the matrix A is $[(\partial l / \partial z_j^{(1)}), (\partial l / \partial z_j^{(2)}), \dots, (\partial l / \partial z_j^{(B)})]$. A set of linear equations is formulated at line 6 and we define vector h_k with B elements as the solution of that linear system. Vector h_k is the representation of B images at hidden neuron k . We repeated encoding reconstruction for other hidden units until $k = n_h$. Finally, the GRA method outputs the representation H of B training images at the final hidden layer.

B. Image Reconstruction Using Image Encoding

Given the network architecture, network parameters θ and the reconstructed representation in the final hidden layer, we find the dummy images $\mathbf{x}^*$ that minimizes the following weighted loss function:

$$\mathbb{L} = \alpha_h L_h + \alpha_{TV} L_{TV} + \alpha_{\text{norm}} L_{\text{norm}} \quad (9)$$

where the mean-squared error L_h represents the difference between the dummy and reconstructed image encoding; the total variation loss L_{TV} is the mean of the difference between two nearby pixels in an image; and the six-norm regularizer $L_{\text{norm}} = \sum (\mathbf{x}^*)^6$ is used to penalize pixels with high values. Only the gradient in the final hidden layer is used for image reconstruction. Later, we evaluate how the gradient in the final layer affects the image reconstruction. In phase 2, we use the L-BFGS optimizer to update each dummy image.

C. Images Reconstruction Using the Whole Shared Gradient

Dummy images obtained from phase 2 are used as the input for phase 3. In this phase, we use the whole shared gradient to reconstruct images. The weighted loss function can be written as

$$\mathbb{L} = \alpha_g L_g + \alpha_h L_h + \alpha_{TV} L_{TV} + \alpha_{\text{norm}} L_{\text{norm}} \quad (10)$$

Algorithm 2 Gradient Attack Algorithm for Batched Data

Input: Network architecture, network parameters (weight, bias), shared gradient wrt parameters ∇W on batched data

- 1: Randomly initialize reconstructed images $\mathbf{x}^*$
- Phase 1: Reconstruction of Image Encoding**
- 2: Derive image encoding h using One-hot Activation or Gradient Average Method
- Phase 2: Image Reconstruction using Image Encoding**
- 3: **for** $i = 1, 2, \dots, B$ **do**
- 4: Compute image encoding for dummy images $h^* = \text{subnet}(\mathbf{x}^*)$
- 5: **for** $\text{epoch} = 1, 2, \dots, n_{\text{epochs}}$ **do**
- 6: $l_h = \text{MSE}(h_i, h_i^*)$
- 7: $l_{TV} = \text{TV}(\mathbf{x}_i^*)$
- 8: $l_{\text{norm}} = \sum (\mathbf{x}_i^*)^6$
- 9: $\mathbb{L} = \alpha_h l_h + \alpha_{TV} l_{TV} + \alpha_{\text{norm}} l_{\text{norm}}$
- 10: Update the image $\mathbf{x}_i^*$ iteratively to minimize $\mathbb{L}$
- 11: **end for**
- 12: **end for**
- Phase 3: Images Reconstruction using the whole Shared Gradient**
- 13: **for** $\text{epoch} = 1, 2, \dots, n_{\text{epochs}}$ **do**
- 14: Compute $\frac{\partial l^*}{\partial \theta}$ with batched images $\mathbf{x}^*$
- 15: $l_{\text{grad}} = \text{MSE}(\frac{\partial l^*}{\partial \theta}, \frac{\partial l}{\partial \theta})$
- 16: $l_h = \sum_i^B \text{MSE}(h_i, h_i^*)$
- 17: $l_{TV} = \sum_i^B \text{TV}(\mathbf{x}_i^*)$
- 18: $l_{\text{norm}} = \sum_i^B \sum (\mathbf{x}_i^*)^6$
- 19: $\mathbb{L} = \alpha_g l_g + \alpha_h l_h + \alpha_{TV} l_{TV} + \alpha_{\text{norm}} l_{\text{norm}}$
- 20: Update all images $\mathbf{x}^*$ iteratively to minimize $\mathbb{L}$
- 21: $\mathbf{x}^* = \text{clip}(\mathbf{x}^*)$
- 22: **end for**

Output: Reconstructed images $\mathbf{x}^*$

where L_g shows the difference between dummy gradient $(\partial l^* / \partial \theta)$ and shared gradient $(\partial l / \partial \theta)$ using mean-squared error; the mean-squared error L_h represents the discrepancy between the dummy and reconstructed image encoding; and the final two regularization terms are used to keep the natural of images. Similar to phase 2, the L-BFGS optimizer is used to find all dummy images that minimize the weighted loss function. Finally, we clip the images into the range $[0, 1]$ to make the results look more natural. Algorithm 2 summarizes the entire procedure of the gradient attack algorithm.

V. PERFORMANCE EVALUATION

We validate our image reconstruction framework *EncGradInversion* for the classification task on the three datasets, including CIFAR-100, LFWPeople, and ImageNet under three different network architectures: 1) LeNet; 2) ResNet-18; and 3) simplified MobileNet. LeNet includes

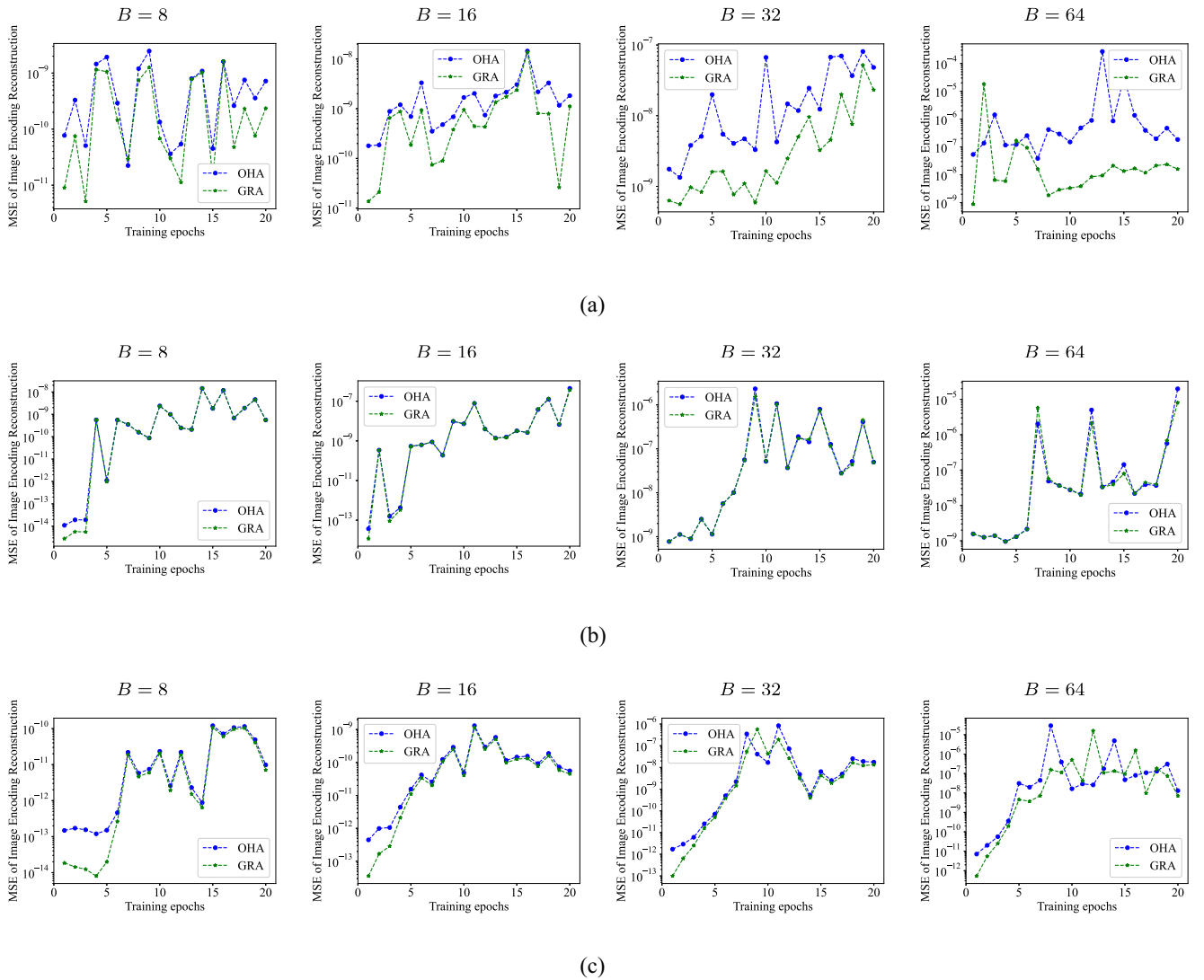


Fig. 3. MSE of OHA and GRA for image encoding reconstruction. (a) CIFAR-100 with 100 classes. (b) LFWPeople with 5749 classes. (c) ImageNet with 1000 classes.

four 5×5 convolutional layers with 12 filters in each convolutional layer followed by a fully dense layer. ResNet-18 consists of a 3×3 kernel in the first convolutional layer. It has four layers of ConvNets with 8, 12, 16, and 24 filters. Each layer consists of two residual blocks and each block consists of two convolutional layers with a skip connection connected to the output of the second convolutional layer with a sigmoid activation function. Meanwhile, the simplified MobileNet has one convolutional layer and three deepwise separable convolution layers. Each deepwise separable convolution layer consists of a deepwise and pointwise convolution. The default image resolution for the CIFAR-100, LFWPeople, and ImageNet datasets are 32×32 , 128×128 , and 256×256 , respectively. The network parameters are randomly initialized using the Kaiming method [21].

We first show the performance comparison between two methods for data encoding reconstruction. The MSE between true and reconstructed image representation is used as a performance metric for data encoding reconstruction. Then, we evaluate the reconstruction quality with different image

resolution and batch size. Five different metrics are used for performance evaluation: MSE, structural similarity index measure (SSIM) [14], peak signal-to-noise ratio (PSNR), cosine similarity in FFT-2D frequency response, and learned perceptual image patch similarity (LPIPS) [10]. Next, a performance comparison between our work and four existing works of inverting gradients (INGs) [8], improved gradient inversion (IGI) [16], STG [10], and DLG [7], in terms of attack time and reconstruction quality is presented followed by a visualization of our work.

A. Performance Comparison Between OHA and GRA for Image Encoding Reconstruction

We consider three datasets with the LeNet network architecture. The batch size is set to 8, 16, 32, and 64, respectively. Fig. 3 shows the MSE value for two methods during the first 20 training epochs. For each iteration, a batch of images is randomly selected for training at the normal device, and the gradient for these batch images is shared with another

TABLE III
MSE COMPARISON BETWEEN OHA AND GRA METHODS

CIFAR-100	$B = 8$	$B = 16$	$B = 32$	$B = 64$
OHA Method	6.84×10^{-10}	2.12×10^{-9}	2.43×10^{-8}	1.54×10^{-5}
GRA Method	4.31×10^{-10}	1.34×10^{-9}	6.95×10^{-9}	9.28×10^{-7}
FWPeople	$B = 8$	$B = 16$	$B = 32$	$B = 64$
OHA Method	2.07×10^{-9}	3.65×10^{-8}	2.68×10^{-7}	1.39×10^{-6}
GRA Method	2.06×10^{-9}	3.31×10^{-8}	2.23×10^{-7}	8.49×10^{-7}
ImageNet	$B = 8$	$B = 16$	$B = 32$	$B = 64$
OHA Method	2.80×10^{-11}	1.78×10^{-10}	7.02×10^{-8}	1.77×10^{-6}
GRA Method	2.45×10^{-11}	1.54×10^{-10}	4.71×10^{-8}	9.41×10^{-7}

TABLE IV
PERFORMANCE OF GRADIENT ATTACK ON THE CIFAR-100 DATASET WITH DIFFERENT NETWORK ARCHITECTURES

Batch Size	LeNet			ResNet18			MobileNet		
B	MSE ↓	SSIM ↑	PSNR ↑	MSE ↓	SSIM ↑	PSNR ↑	MSE ↓	SSIM ↑	PSNR ↑
1	0.0056	0.779	23.38	0.0186	0.804	18.41	0.0061	0.769	22.70
2	0.0064	0.783	22.77	0.0171	0.816	19.56	0.0076	0.758	21.72
4	0.0084	0.747	22.01	0.0202	0.779	18.57	0.0109	0.695	20.36
8	0.0099	0.735	21.42	0.0203	0.757	18.53	0.0140	0.653	19.24
16	0.0106	0.739	21.42	0.0343	0.681	17.15	0.0199	0.585	17.91
32	0.0132	0.707	20.62	0.0417	0.621	15.77	0.0370	0.452	15.44

TABLE V
PERFORMANCE OF GRADIENT ATTACK ON THE LFWPEOPLE DATASET WITH DIFFERENT NETWORK ARCHITECTURES

Batch Size	LeNet			ResNet18			MobileNet		
B	MSE ↓	SSIM ↑	PSNR ↑	MSE ↓	SSIM ↑	PSNR ↑	MSE ↓	SSIM ↑	PSNR ↑
1	0.0060	0.818	22.40	0.0026	0.930	27.16	0.0050	0.850	23.19
2	0.0067	0.798	22.18	0.0029	0.926	26.28	0.0057	0.828	22.68
4	0.0061	0.799	22.60	0.0034	0.918	25.54	0.0063	0.805	22.35
8	0.0065	0.790	22.43	0.0034	0.903	25.51	0.0090	0.765	20.95
16	0.0073	0.784	22.25	0.0109	0.840	24.18	0.0156	0.702	19.05
32	0.0079	0.785	22.30	0.0087	0.866	24.21	0.0290	0.628	16.61

TABLE VI
PERFORMANCE OF GRADIENT ATTACK ON THE IMAGENET DATASET WITH DIFFERENT NETWORK ARCHITECTURES

Batch Size	LeNet			ResNet18			MobileNet		
B	MSE ↓	SSIM ↑	PSNR ↑	MSE ↓	SSIM ↑	PSNR ↑	MSE ↓	SSIM ↑	PSNR ↑
1	0.0025	0.892	27.70	0.0202	0.745	19.19	0.0031	0.809	25.76
2	0.0023	0.884	27.73	0.0182	0.769	19.20	0.0031	0.809	25.51
4	0.0025	0.869	27.09	0.0243	0.776	18.60	0.0039	0.817	24.98
8	0.0027	0.866	27.09	0.0256	0.732	17.55	0.0059	0.765	23.14
16	0.0027	0.872	27.40	0.0280	0.706	16.95	0.0100	0.708	20.87
32	0.0025	0.875	27.57	0.0296	0.730	16.51	0.019	0.674	18.58

device that is compromised by the adversary. In most cases, the MSE tends to increase over the training epoch because the gradient related to nontrue outputs likely approximates 0 at the beginning of training. Table III shows the average MSE of two methods during the 20 epochs. Across all of the datasets, we have observed that the gap between the true and reconstructed image encodings tends to be higher with a larger batch size in both methods. For example, in the OHA method, when B jumps from 8 to 64, the reconstruction of image representation deteriorates 6800, 885, and 36 643 times in CIFAR-100, LFWPeople, and ImageNet, respectively.

The GRA method clearly outperforms OHA, especially as the batch size increases. Recall that in OHA, we assume that $(e^{z_j}/[\sum_j e^{z_j}]) \approx 0$ when output j is not the image label and $1 - (e^{z_j}/[\sum_j e^{z_j}]) \approx 1$ with j is the image label. The OHA assumption is more likely true with a higher number

of classes and at the beginning of the training. In GRA, on the other hand, $(e^{z_j}/[\sum_j e^{z_j}]) \approx (1/C)$, which is closer to the real values of $(e^{z_j}/[\sum_j e^{z_j}])$. Therefore, the GRA produces image encoding reconstruction with higher accuracy than OHA, and the GRA is used as our default method for image reconstruction for other experiments.

B. Performance Evaluation for Image Batch Reconstruction

For the experiments in this section, the batch size is set from 1 to 32 and three metrics of MSE, SSIM, and PSNR are used. Tables IV–VI show the results of the CIFAR-100, LFWPeople, and ImageNet datasets, respectively, under three different network architectures of LeNet, ResNet18, and simplified MobileNet. As the batch size increases, performance results gradually deteriorate since the labels of the image batch may

TABLE VII
COMPARISON OF GRADIENT ATTACK'S PERFORMANCE WITH THE LeNet MODEL AND THE CIFAR-100 DATASET

Batch Size B	Performance after phase 2			Performance after phase 3		
	MSE ↓	SSIM ↑	PSNR ↑	MSE ↓	SSIM ↑	PSNR ↑
1	0.0058 ± 0.0032	0.8102 ± 0.0531	23.15 ± 2.93	0.0128 ± 0.0232	0.7622 ± 0.2091	22.25 ± 4.96
2	0.0071 ± 0.0020	0.7929 ± 0.0531	22.19 ± 1.45	0.0140 ± 0.0235	0.7545 ± 0.1700	21.45 ± 3.92
4	0.0060 ± 0.0053	0.8213 ± 0.0431	23.52 ± 2.79	0.0115 ± 0.0175	0.7661 ± 0.1958	22.43 ± 4.20
8	0.0070 ± 0.0027	0.8026 ± 0.0311	22.83 ± 1.40	0.0070 ± 0.0027	0.8028 ± 0.0311	22.83 ± 1.40
16	0.0130 ± 0.0059	0.7551 ± 0.0348	22.15 ± 0.99	0.0130 ± 0.0059	0.7551 ± 0.0348	21.15 ± 0.99
32	0.0188 ± 0.0068	0.7195 ± 0.0348	20.42 ± 1.06	0.0188 ± 0.0068	0.7196 ± 0.0347	20.42 ± 1.06

TABLE VIII
COMPARISON OF GRADIENT ATTACK'S PERFORMANCE WITH THE ResNet18 MODEL AND THE CIFAR-100 DATASET

Batch Size B	Performance after phase 2			Performance after phase 3		
	MSE ↓	SSIM ↑	PSNR ↑	MSE ↓	SSIM ↑	PSNR ↑
1	0.0223 ± 0.0119	0.7662 ± 0.0577	17.02 ± 2.18	0.0178 ± 0.0115	0.8282 ± 0.0933	18.31 ± 2.91
2	0.0253 ± 0.0142	0.7343 ± 0.0945	16.78 ± 2.06	0.0193 ± 0.0101	0.8085 ± 0.0514	18.23 ± 1.88
4	0.0295 ± 0.0128	0.7072 ± 0.0725	16.39 ± 1.89	0.0245 ± 0.0113	0.7673 ± 0.0576	17.49 ± 2.09
8	0.0295 ± 0.0077	0.6918 ± 0.0629	16.29 ± 1.11	0.0259 ± 0.0082	0.7272 ± 0.0708	17.09 ± 1.25
16	0.0264 ± 0.0041	0.7232 ± 0.0393	16.65 ± 0.57	0.0250 ± 0.0037	0.6842 ± 0.0364	16.73 ± 0.58
32	0.0348 ± 0.0077	0.6383 ± 0.0687	15.74 ± 0.793	0.0351 ± 0.0067	0.5655 ± 0.0582	15.42 ± 0.63

TABLE IX
PERFORMANCE COMPARISON WITH OTHER COUNTERPARTS

Batch Size =1	MSE ↓	SSIM ↑	PNSR ↑	FFT-2D ↓	LPIPS ↓
DLG [7]	0.0018	0.942	27.39	0.978	0.036
STG [10]	0.0067	0.749	22.32	0.930	0.17
ING [8]	0.0075	0.782	22.25	0.955	0.077
IGI [16]	0.0017	0.908	28.34	0.980	0.050
<i>EncGradInversion</i>	0.0056	0.779	23.38	0.934	0.147
Batch Size =2	MSE ↓	SSIM ↑	PNSR ↑	FFT-2D ↓	LPIPS ↓
DLG [7]	0.1829	0.096	8.381	0.439	0.507
STG [10]	0.012	0.667	19.83	0.895	0.235
ING [8]	0.0155	0.723	18.57	0.919	0.110
IGI [16]	0.0096	0.626	20.61	0.912	0.204
<i>EncGradInversion</i>	0.0064	0.783	22.77	0.933	0.147
Batch Size =4	MSE ↓	SSIM ↑	PNSR ↑	FFT-2D ↓	LPIPS ↓
DLG [7]	0.2695	0.029	6.08	0.405	0.540
STG [10]	0.0198	0.536	17.84	0.832	0.336
ING [8]	0.0549	0.422	13.31	0.779	0.272
IGI [16]	0.074	0.473	16.42	0.766	0.273
<i>EncGradInversion</i>	0.0086	0.747	22.01	0.916	0.153
Batch Size =8	MSE ↓	SSIM ↑	PNSR ↑	FFT-2D ↓	LPIPS ↓
DLG [7]	0.25	0.075	6.69	0.455	0.534
STG [10]	0.153	0.126	8.69	0.518	0.549
ING [8]	0.109	0.225	10.02	0.654	0.415
IGI [16]	0.087	0.378	14.852	0.717	0.327
<i>EncGradInversion</i>	0.010	0.735	21.42	0.914	0.151

likely be duplicated. When there are more images with the same label, reconstruction of image encoding becomes more difficult, thus causing lower accuracy for image reconstruction.

Note that, the LFWPeople dataset has 5749 classes compared to 100 in the CIFAR-100 and 1000 in ImageNet. Moreover, CIFAR-100, LFWPeople, and ImageNet datasets contain 50k, 13k, and 1k images, or 500, 2.2, and 1 image per each class, respectively. For the evaluation, we assume that the normal participant randomly selects B images in the whole dataset for model training, and then the victim shares the gradient with other devices. The effects of B are less seen in the ImageNet dataset compared to the other datasets since it

has only one thousand images in the training dataset. As shown in Table VI, with the ResNet18 architecture, SSIM and PSNR values drop only 2% and 14%, respectively, as B changes from 1 to 32. In contrast, we achieved the lowest performance for image reconstruction in the CIFAR-100 dataset due to the high probability of having images with duplicate labels in the image batch. For example, when using the simplified MobileNet, the MSE increases 6.1 times from 0.0061 to 0.0370, while SSIM and PSNR drop 41.2% and 31.9%, respectively, when the batch size increases from 1 to 32.

Next, we show the performance comparison of gradient attack at phases 2 and 3 in *EncGradInversion* using two

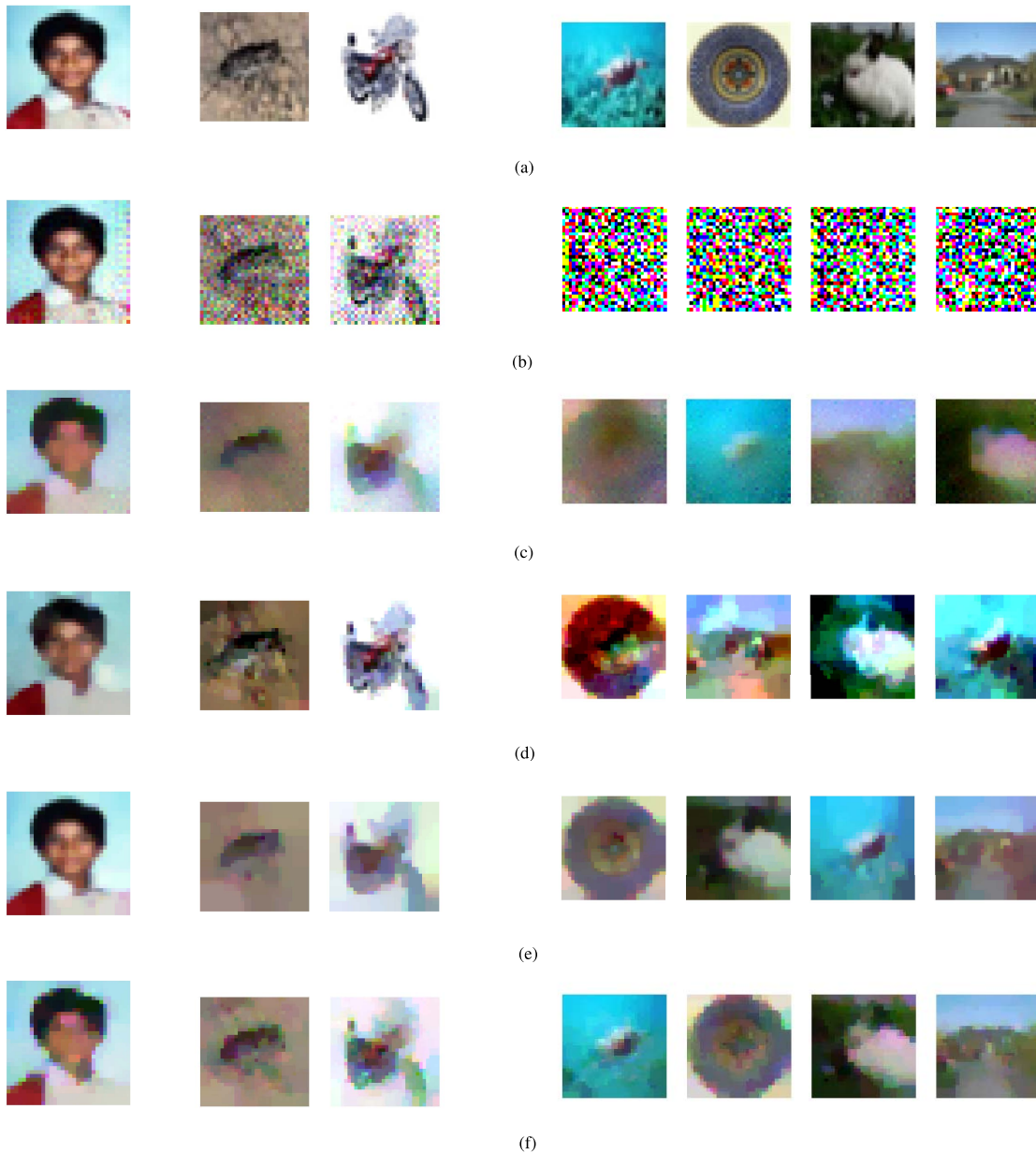


Fig. 4. Comparison between image reconstruction. (a) Original batch. (b) DLG. (c) STG. (d) ING. (e) IGI.

network architectures, e.g., LeNet in Table VII and ResNet18 in Table VIII on the CIFAR-100 dataset. We perform experimental results with ten runs and present the mean and standard derivation of the results. Note that, we only use the shared gradients in the final hidden layer for image reconstruction in phase 2, whereas the shared gradients from the whole network are used in phase 3. Basically, since phase 3 requires more information for gradient attack, we aim to check how phase 3 helps the adversary recover the training images. The experimental results show that the shared gradients of the whole network can be leveraged to enhance the performance better especially as the batch size is small. Specifically, Table VIII shows that the reconstruction performance of *EncGradInversion* after phase 3 is better than after phase 2 in terms of MSE, SSIM, and PSNR when $B \leq 16$. When B increases, the number of parameters that need to be optimized

increases, leading to more difficulty in recovering the training images. That is why Table VIII shows no improvement at phase 3 when $B > 16$ in the ResNet18.

With the LeNet model, we observe that performance at phase 2 is slightly better than that at phase 3. Therefore, we present the results in phase 2 with more reduced attack time for other experiments. Without phase 2, our *EncGradInversion* algorithm leverages the shared gradient of all layers for image reconstruction, which is similar to the DLG method [7]. The performance comparison between our method and DLG is presented in the following section.

C. Performance Comparison With Existing Studies on Image Batch Reconstruction

We selected four related works for performance comparison: 1) ING [8]; 2) IGI [16]; 3) STG [10]; and 4) DLG [7]. In

these methods, they formulated an optimization problem with variables of dummy images. For a fair comparison, dummy images are randomly initialized and there is no auxiliary set at the adversary. In IGI, DLG, and STG methods, the gradient loss is defined as the squared difference between shared and dummy gradients. Meanwhile, the ING algorithm considers cosine similarity between shared and dummy gradients as the gradient loss. There is no regularizer in DLG; in contrast, ING includes a total variation loss. Meanwhile, regularization terms of total variation loss, norm loss, and group consistency are added to the loss function in STG. Finally, IGI consists of total variation, clip, and scale losses. For the experiment setup, B is set to $\{1, 2, 4, 8\}$, and five different metrics are collected for performance comparison. We consider the LeNet network and CIFAR-100 dataset for this experiment. We present the average results of reconstructing ten different image batches.

With $B = 1$, all of the methods successfully recover the training image, which indicates that it is not difficult to launch the gradient attack with a single training image. When $B > 1$, DLG has the lowest reconstruction performance with regard to MSE, SSIM, PSNR, FFT-2D, and LPIPS. For example, the MSE, SSIM, PSNR, FFT-2D, and LPIPS of DLG are 0.2695, 0.029, 6.08, 0.405, and 0.540, respectively, when $B = 4$. Thanks to the regularization terms, STG, ING, and IGI achieve a higher performance than DLG. IGI produces higher reconstruction accuracy than STG and ING, highlighting the significance of clip and scale losses. However, our *EncGradInversion* method generally outperforms STG, ING, IGI, especially with $B > 1$. As B increases, the performance gap rather increases as shown in Table IX. For example, the SSIM difference between our work and STG is 0.116 with $B = 2$ and 0.609 with $B = 8$, respectively. The superior performance of *EncGradInversion* compared to other methods clearly shows the importance of image representation in image reconstruction. We also visualize the reconstructed images of our work and other counterparts compared with the original images in Fig. 4.

Now, we evaluate the attack time of our method, DLG, STG, IGI, and ING in Fig. 5. The LeNet architecture and the CIFAR-100 dataset are used for this experiment. As mentioned earlier, existing works usually suffer from performance degradation for large batch sizes. In contrast, since each training image is reconstructed from its image representation in phase 2, our work is less affected by the batch size. As the batch size changes from 1 to 512, it generally takes more time to launch the gradient attack in all attack strategies. However, our proposed method always has the lowest attack time. The performance gain keeps increasing as B increases. For example, our method reduces 2.3 and 13.4 s with $B = 1$, and 1210 and 5152 s with $B = 512$ compared to DLG and STG, respectively. From results presented in this section, we can conclude that *EncGradInversion* surpasses other counterparts in both reconstruction quality and attack time.

D. Visualization

We demonstrate the quality of the reconstructed images by our *EncGradInversion* (with $B = 32$, compared to the

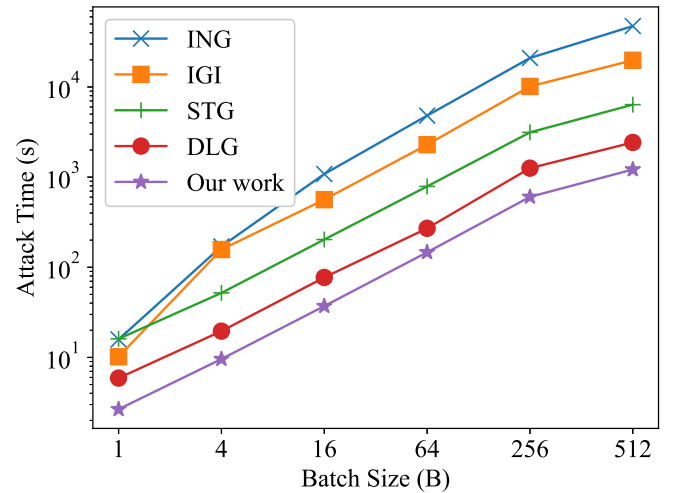


Fig. 5. Comparison of attack time between gradient attack methods.

original training images in three datasets in Fig. 6. In CIFAR-100, the image resolution is set to 32×32 , and a specific image has the average of 500 other images with the same class, compared to 2.2 and 1 in LFWPeople and ImageNet, respectively. If there exist two images with the same label in the image batch, the image recovery algorithm only results in a combination of both images, as shown in Fig. 6(a). Since the probability of having images with duplicate labels is much less than CIFAR-100, and the image resolution is higher than CIFAR-100, the reconstruction quality in LFW or the ImageNet dataset is much better than CIFAR-100. We can infer from Fig. 6 that as long as a specific image has no duplicate label, this image can be accurately recovered through *EncGradInversion*, which can pose a real feasible threat to data privacy of participants.

Finally, we show the impacts of image resolution on the gradient-based reconstruction attack with the ImageNet dataset in Fig. 7. The image size changes from 32×32 to 1024×1024 , and the batch size is set to 4. Normally, when using higher image resolution, our *EncGradInversion* achieves better classification accuracy, and the training images can also be better recovered, which leads to a higher threat to distributed learning. It should be noted that the other methods of gradient attacks can also successfully recover the training images with lower resolution since there are fewer variables that can easily be optimized in the optimization problem.

E. Discussion

We discuss some important main findings and implications of this work below.

- 1) *EncGradInversion* utilizes image encoding and the shared gradient to achieve higher reconstruction performance than state-of-the-art gradient attacks, as measured by MSE, SSIM, PSNR, FFT-2D, and LPIPS. Experimental results indicate that employing the whole shared gradient yields more effective image reconstruction when working with a small batch size, while using only image encoding is recommended for larger batch sizes.



Fig. 6. Original images (left) and reconstructed images (right) by our *EncGradInversion* method with different datasets. (a) CIFAR-100. (b) LFW. (c) ImageNet.

- 2) *EncGradInversion* successfully reconstructs training images with the resolution as high as 1024×1024 and the batch size of 512, which is much higher than other existing works. In the line with related works, our results show that superior reconstruction performance has been achieved with a lower image resolution, a smaller batch size, or fewer images with duplicate labels. Moreover, a model with random parameters is more susceptible to the gradient attack than the well-trained model.
- 3) *EncGradInversion* is able to recover training images in parallel, leading to substantial reduction in attack time. Our work is the first attempt to reconstruct batch images independently using the image encoding extracted at the final hidden layer.

VI. CONCLUSION

We introduced a novel gradient attack mechanism *EncGradInversion* that leverages the fusion of image encoding

and the shared gradient to effectively reconstruct training images in FL. *EncGradInversion* operates through three phases: 1) image encoding reconstruction; 2) image reconstruction using image encoding; and 3) image reconstruction using the whole shared gradient. Thanks to image encoding, *EncGradInversion* supports image reconstruction simultaneously, validating particularly advantageous when dealing with large batch sizes. Training images can be derived by matching both image encoding and the shared gradient between dummy and real training images, while preserving the realism of reconstructed images. Experimental results show that *EncGradInversion* leads to higher reconstruction quality than two related works and significant reduction in attack time. As future work, we may investigate the gradient attack in a more challenging situation involved with the utilization of multiple images with the identical labels in the batch and the application of transfer learning at each client.

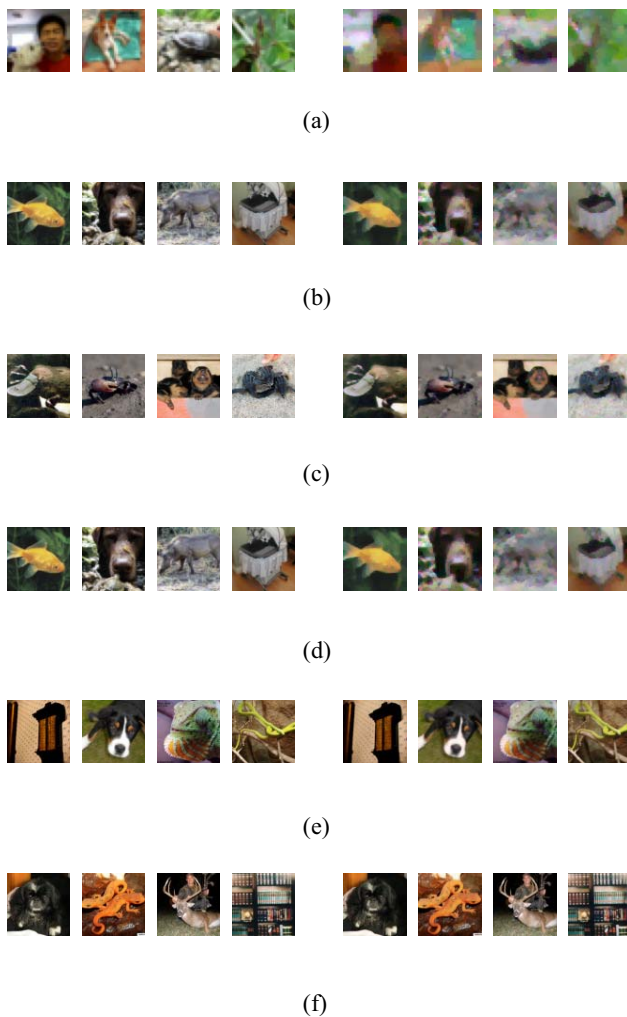


Fig. 7. Impacts of image resolution on the image reconstruction: original images (left) and reconstructed images (right) by our *EncGradInversion* method on the ImageNet dataset. (a) 32×32 . (b) 64×64 . (c) 128×128 . (d) 256×256 . (e) 512×512 . (f) 1024×1024 .

REFERENCES

- [1] Q. Li et al., "A survey on federated learning systems: Vision, hype and reality for data privacy and protection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 3347–3366, Apr. 2023.
- [2] S. AbdulRahman, H. Tout, H. Ould-Slimane, A. Mourad, C. Talhi, and M. Guizani, "A survey on federated learning: The journey from centralized to distributed on-site learning and beyond," *IEEE Internet Things J.*, vol. 8, no. 7, pp. 5476–5497, Apr. 2021.
- [3] N. Rieke et al., "The future of digital health with federated learning," *NPJ Digit. Med.*, vol. 3, no. 1, p. 119, 2020.
- [4] T. Liu et al., "Efficient and secure federated learning for financial applications," *Appl. Sci.*, vol. 13, no. 10, p. 5877, 2023.
- [5] F. A. KhoKhar, J. H. Shah, M. A. Khan, M. Sharif, U. Tariq, and S. Kadry, "A review on federated learning towards image processing," *Comput. Electr. Eng.*, vol. 99, 2022, Art. no. 107818.
- [6] S. Nazir and M. Kaleem, "Federated learning for medical image analysis with deep neural networks," *Diagnostics*, vol. 13, no. 9, p. 1532, 2023.
- [7] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 14774–14784.
- [8] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients—how easy is it to break privacy in federated learning?" in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 16937–16947.
- [9] J. Zhu and M. Blaschko, "R-GAP: Recursive gradient attack on privacy," 2020, *arXiv:2010.07733*.

- [10] H. Yin, A. Mallya, A. Vahdat, J. M. Alvarez, J. Kautz, and P. Molchanov, "See through gradients: Image batch recovery via gradient inversion," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 16337–16346.
- [11] L. Fowl, J. Geiping, W. Czaja, M. Goldblum, and T. Goldstein, "Robbing the fed: Directly obtaining private data in federated learning with modified models," 2021, *arXiv:2110.13057*.
- [12] A. Hatamizadeh et al., "Do gradient inversion attacks make federated learning unsafe?" *IEEE Trans. Med. Imag.*, vol. 42, no. 7, pp. 2044–2056, Jul. 2023.
- [13] F. Boenisch, A. Dziedzic, R. Schuster, A. S. Shamsabadi, I. Shumailov, and N. Papernot, "When the curious abandon honesty: Federated learning is not private," in *Proc. IEEE 8th Eur. Symp. Security Privacy (EuroSP)*, 2023, pp. 175–199.
- [14] H. Liang, Y. Li, C. Zhang, X. Liu, and L. Zhu, "EGIA: An external gradient inversion attack in federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 4984–4995, 2023.
- [15] H. Yang, M. Ge, K. Xiang, and J. Li, "Using highly compressed gradients in federated learning for data reconstruction attacks," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 818–830, 2022.
- [16] J. Geng et al., "Improved gradient inversion attacks and defenses in federated learning," *IEEE Trans. Big Data*, early access, Jan. 23, 2023, doi: [10.1109/TBDATA.2023.3239116](https://doi.org/10.1109/TBDATA.2023.3239116).
- [17] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [18] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.
- [19] A. Mahendran and A. Vedaldi, "Understanding deep image representations by inverting them," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 5188–5196.
- [20] B. Zhao, K. R. Mopuri, and H. Bilen, "iDLG: Improved deep leakage from gradients," 2020, *arXiv:2001.02610*.
- [21] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2015, pp. 1026–1034.



Thi-Nga Dao received the B.S. degree in electrical and communication engineering from Le Quy Don Technical University, Hanoi, Vietnam, in 2013, and the M.S. and Ph.D. degrees in computer engineering from the University of Ulsan, Ulsan, South Korea, in 2016 and 2019, respectively.

She was a Postdoctoral Fellow with the Department of Computer Science and Engineering, Ewha Womans University, Seoul, South Korea. Since July 2019, she has been a Lecturer with the Faculty of Radio-Electronic Engineering, Le Quy Don Technical University. Her research interests include machine learning-based applications in network security, network intrusion detection and prevention systems, human mobility prediction, and mobile crowdsensing.



Hyungjune Lee (Member, IEEE) received the B.S. degree in electrical engineering from Seoul National University, Seoul, South Korea, in 2001, and the M.S. and Ph.D. degrees in electrical engineering from Stanford University, Stanford, CA, USA, in 2006 and 2010, respectively.

He joined Broadcom, Santa Clara, CA, as a Senior Staff Scientist for working on research and development of 60 GHz 802.11ad SoC MAC. He worked with AT&T Labs, San Ramon, CA, as a Principal Member of Technical Staff with the involvement of LTE overload estimation, LTE-WiFi interworking, and heterogeneous networks. He is currently a Professor with the Computer Science and Engineering Department, Ewha Womans University, Seoul. His current research interests include distributed learning and future wireless networks on the IoT, fog computing, VANET, and machine learning-driven network system design.